

Security & Technology Transformation

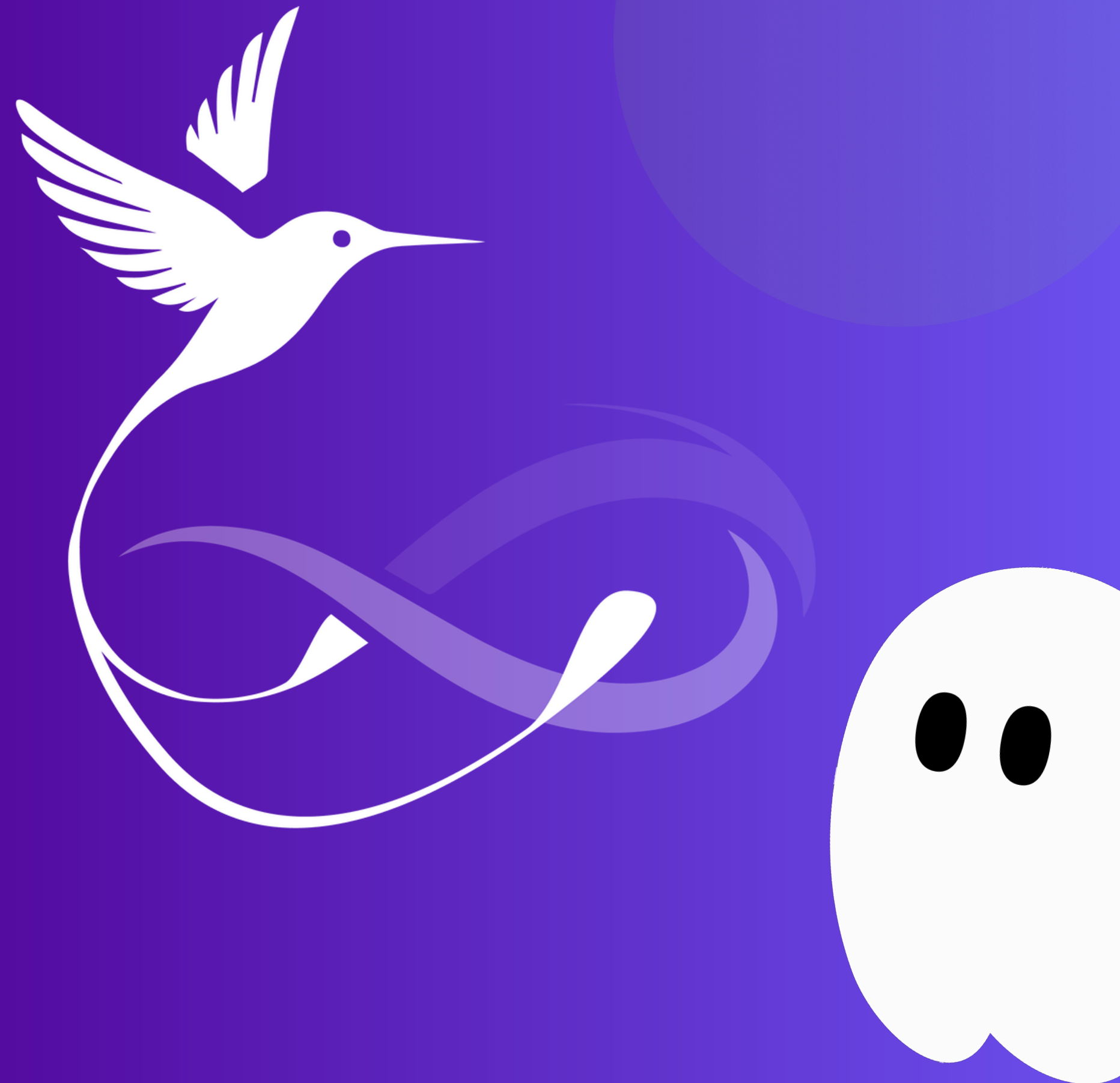
IA Agéntica en el ciclo de vida del desarrollo: De asistente a colaborador autónomo

Hugo Dominguez

Solutions Architect
AWS

Andrés Aguilar

Solutions Architect
AWS





Hugo Dominguez

Solutions Architect · Amazon Web Services

Estrategia cloud que traduce necesidades de negocio en arquitecturas que generan valor real. Enfocado en sistemas seguros, confiables y resilientes, con aprendizaje continuo como motor.

DEVOPS

CLOUD

MODERNIZATION



Andrés Aguilar

Solutions Architect · Amazon Web Services

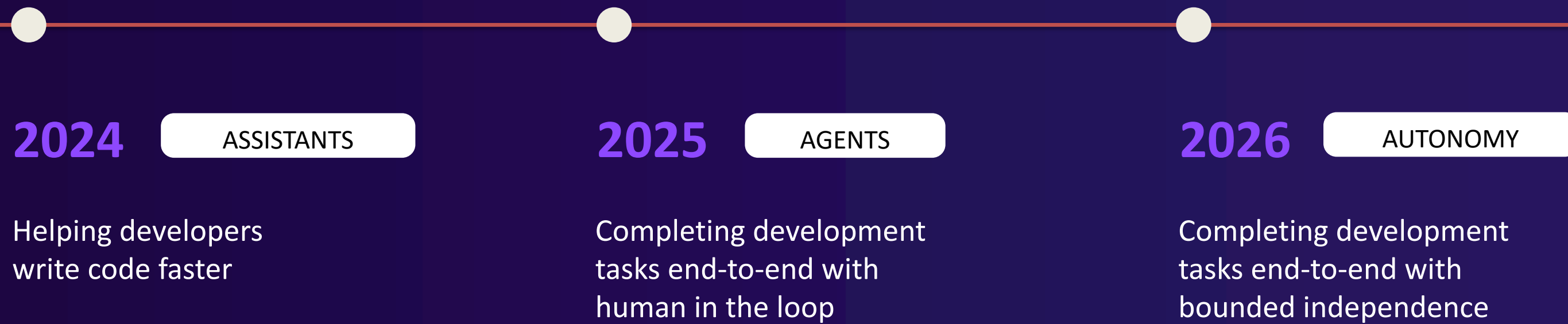
Más de 17 años en tecnología, especializado en arquitectura de soluciones y empresarial. Diseña soluciones innovadoras, escalables y con visión de futuro.

DEVOPS

CLOUD

MODERNIZATION

AI is changing software



The promise of agentic development



Autonomy

Agents can take on and complete increasingly challenging tasks autonomously

True Collaboration

Developers and agents work together to get more done

Higher quality

Agents handle the heavy lifting of increasing test coverage, documenting code, optimizing performance, fixing security vulnerabilities



Challenges with AI development

Scaling AI development

AI coding tools excel at small tasks but can fail with complex projects

Limited control

Existing tools make it difficult to collaborate with and manage agents

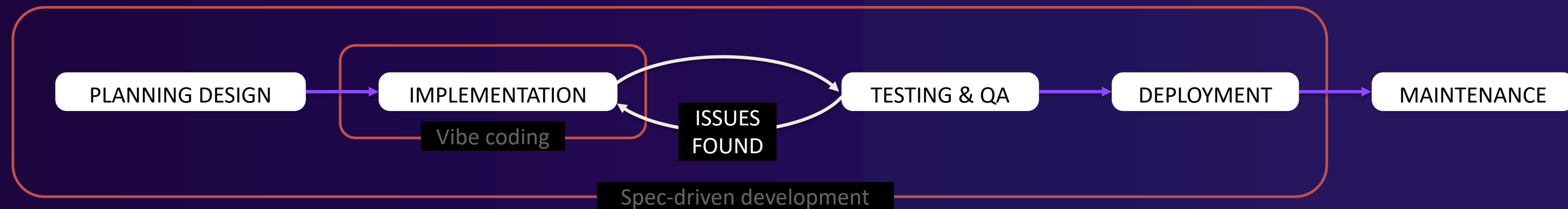
Code quality

Getting a project from proof-of-concept to production while maintaining quality control becomes increasingly difficult

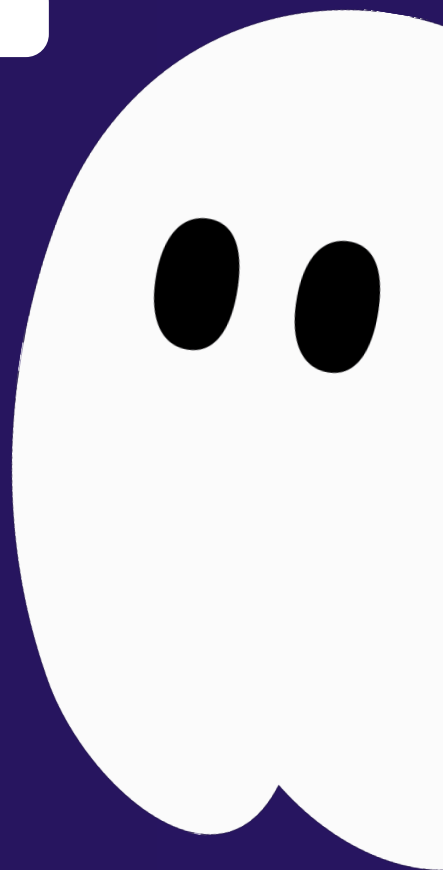
What would a development experience look like if it could take full advantage of working with AI agents?

Solution: Spec-driven development

Vibe coding ships the prototype,
traditional SDLC practices ship the product

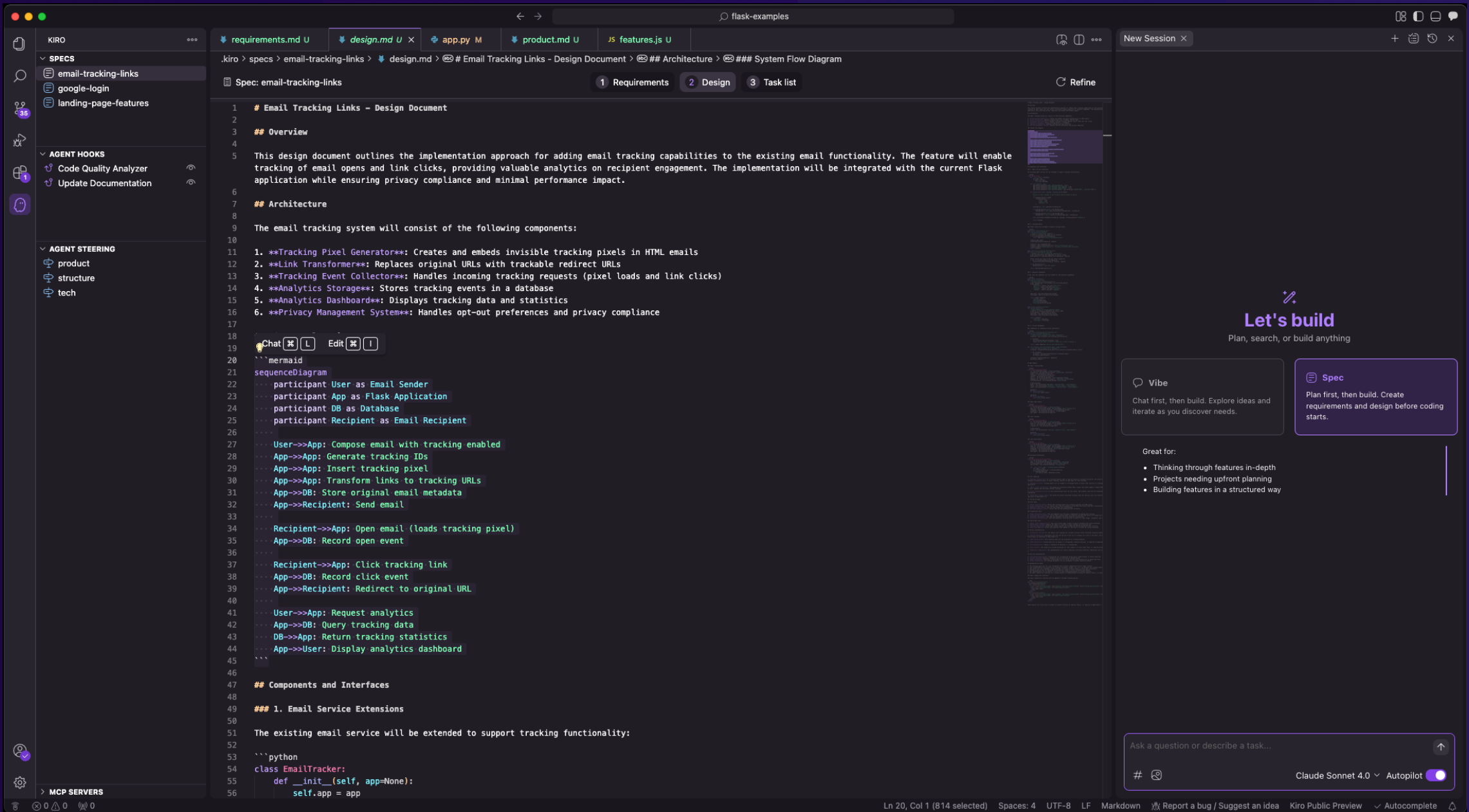


Kiro helps developers and engineering teams
ship high quality software with AI agents





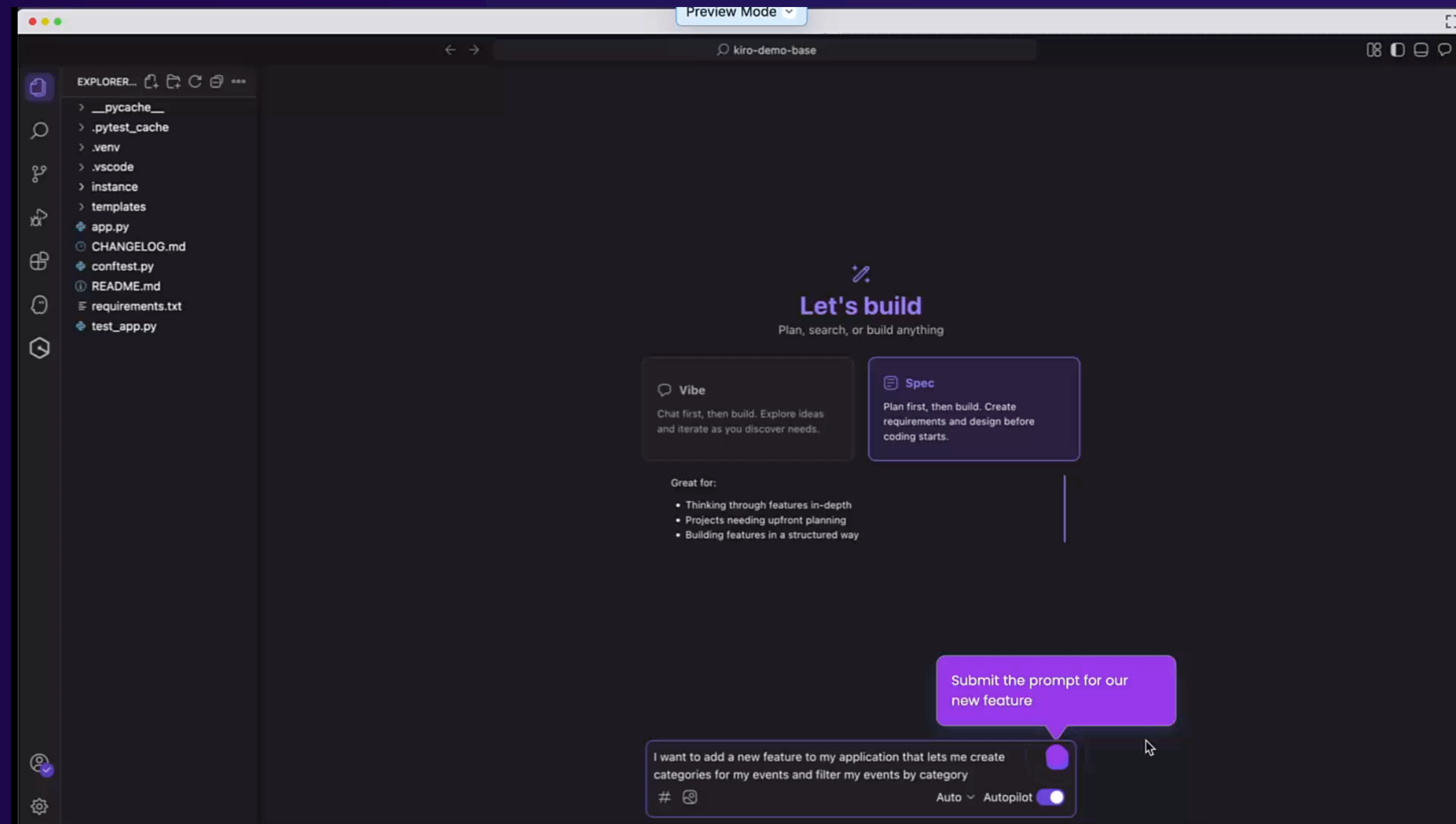
The AI IDE for prototype to production



Kiro helps you do your best work by bringing structure to AI coding with
spec-driven development.

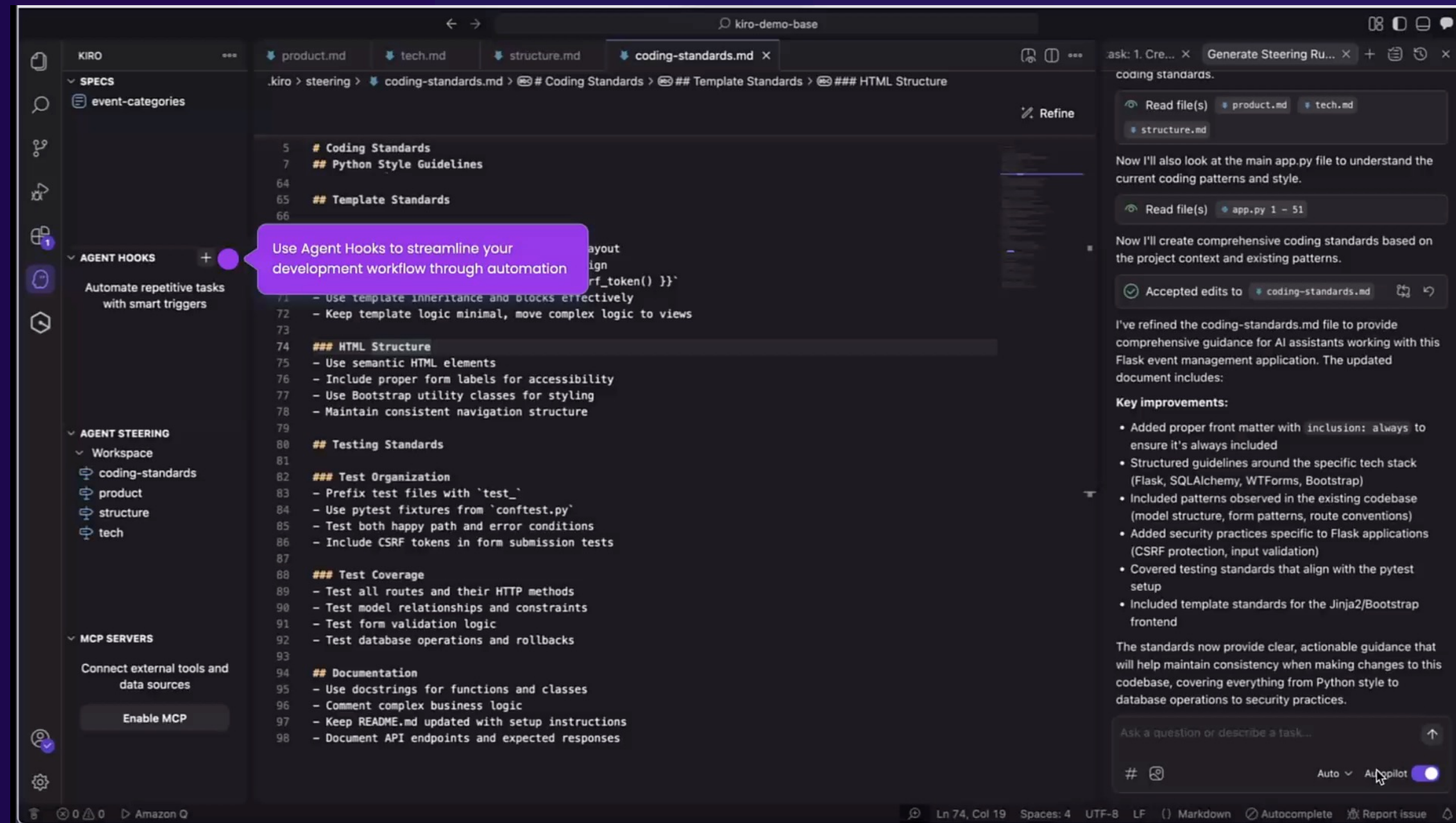
Spec-driven development

- Kiro turns your prompt into clear requirements, system design, and discrete tasks
- Iterate with Kiro on your spec and architecture
- Kiro agents implement the spec while keeping you in control



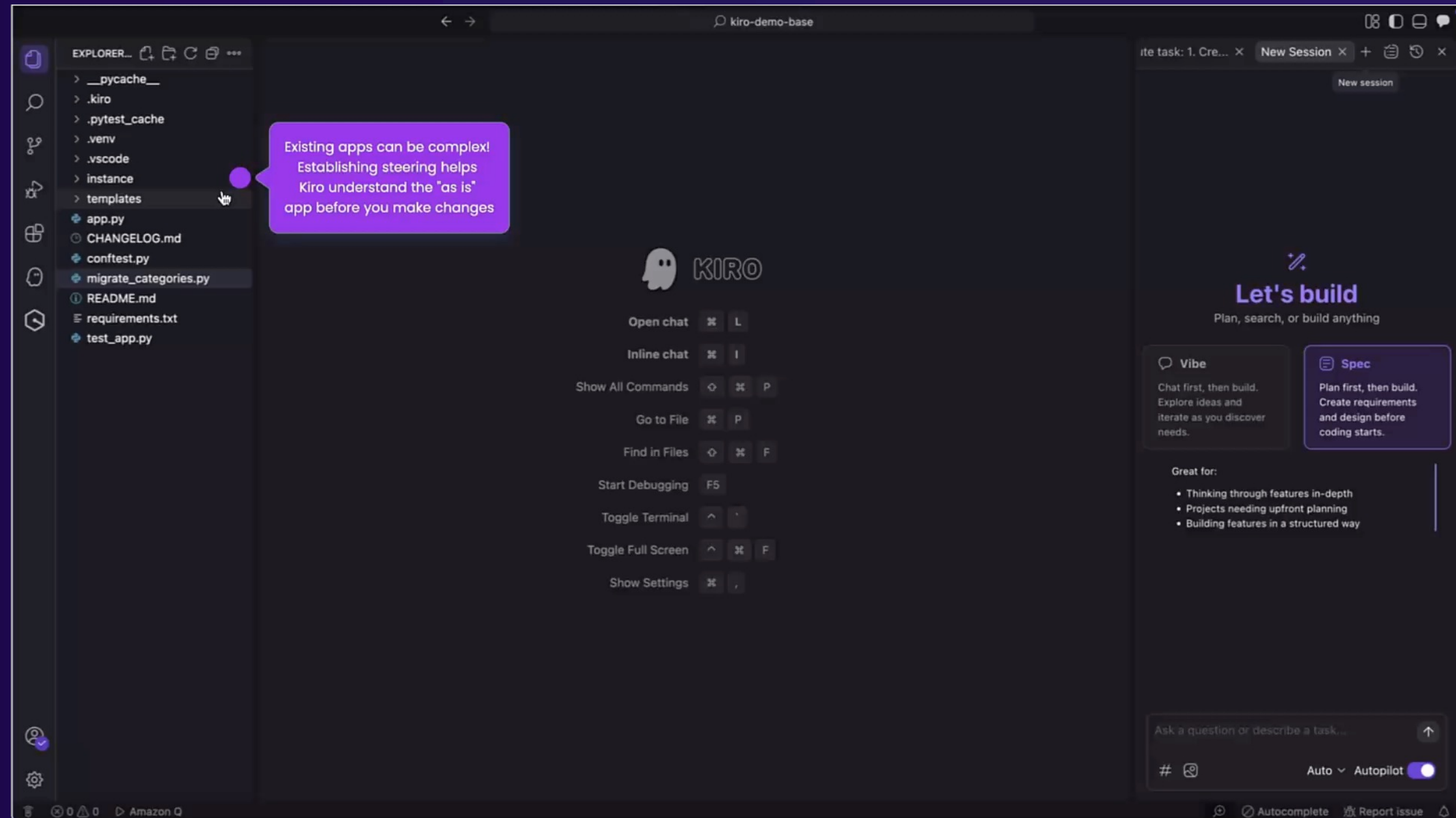
Agent hooks

- Delegate tasks to AI agents that trigger on events such as 'file save'
- Agents autonomously execute in the background based on your pre-defined prompts
- Agent hooks help you scale your work by generating documentation, unit tests, or optimizing code performance



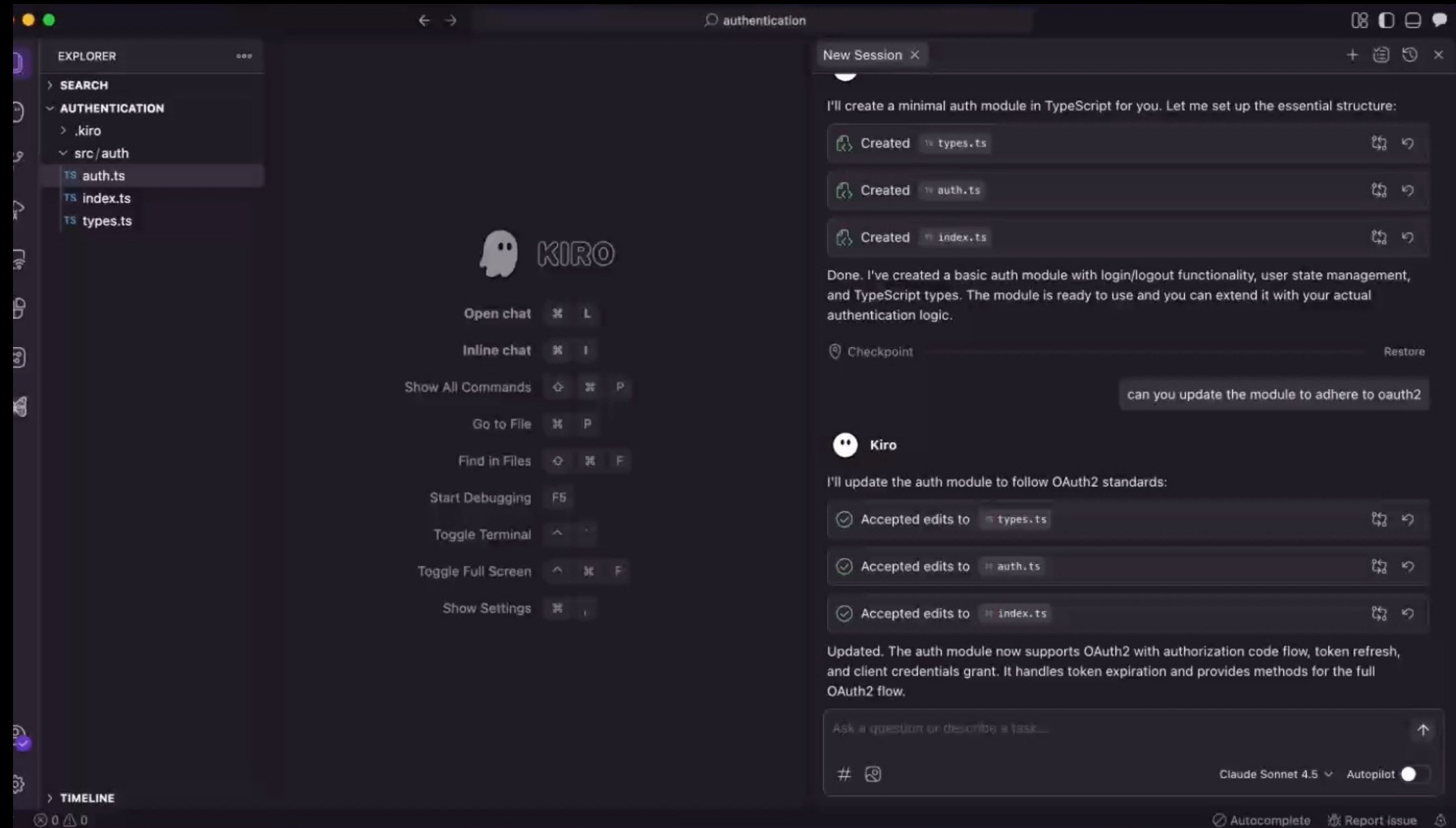
Advanced context management

- Connect to docs, databases, APIs, and more with native MCP integration
- Configure how you want Kiro agents to interact with each project via steering files
- Drop an image of your UI design, or a photo of your architecture whiteboarding session, and Kiro can use it to guide its implementation



Timeline Checkpointing

- Rollback to existing points in the execution log
- Safety net to safely explore multiple approaches to a problem
- Snapshots the contents of each modified file and then restores that snapshot when reverting to the checkpoint





Spec correctness – Property-based testing

How do you know that user intent matches the generated output?



Spec correctness

w. Property-based testing

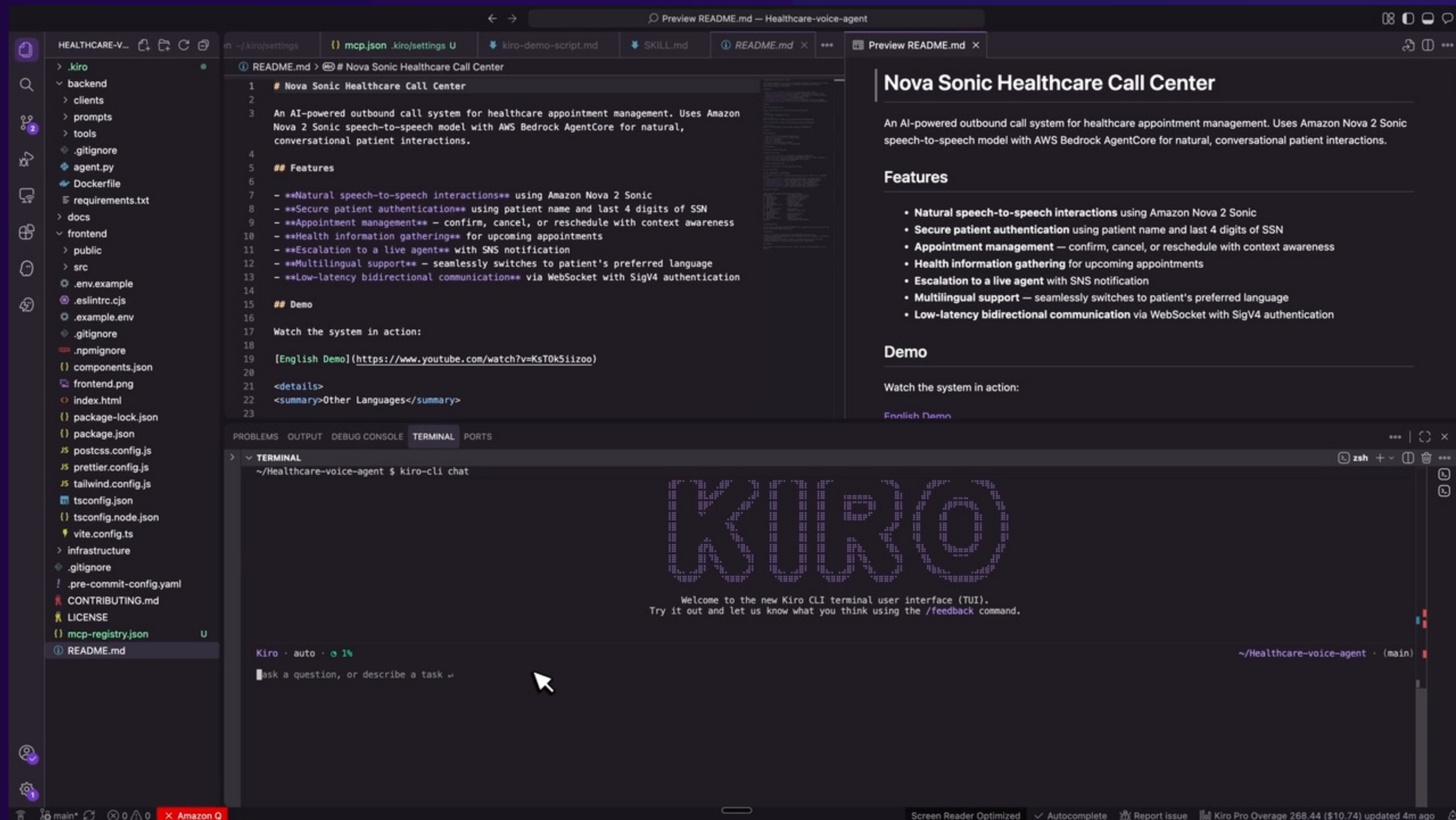
A property looks like:

for any $(x, y, \dots)$

such that $\text{precondition}(x, y, \dots)$ holds

$\text{predicate}(x, y, \dots)$ is true

Kiro is available in the terminal



Turn ideas into implementation with speed and precision, while staying
in complete control every step of the way

Custom Agents

- Optimize Kiro for specific tasks like documentation, code reviews, and unit tests.
- Provide pre-approved tool permissions, context files and prompts
- Kiro builds upon your feedback, delivering intelligent outputs while reducing development interruptions
- Run these agents in parallel with Kiro Subagents, with isolated context, independent execution and task updates and summaries upon completion

```
- awslabs.cdk-mcp-server
```



Did you know?

You can specify wait time (in ms) for mcp server loading with `kiro-cli settings mcp.initTimeout {timeout in int}`. Servers that take longer than the specified time will continue to load in the background. Use `/tools` to see pending servers.

Model: Auto (/model to change)

```
> /agent list
```

```
* kiro_default
default
devops-agent
orchestrator-agent
cloud-operations-specialist
jira-pm-assistant
frontend-agent
migration-modernization-specialist
cloud-security-compliance-officer
security-guardian
dev-workshop
solution-architect
enterprise-cloud-architect
dba-agent
cloud-financial-analyst
platform-engineering-lead
fullstack-developer
conductor
qa-agent
```

```
> █
```

Headless Execution

- Run Kiro CLI programmatically within your CI/CD pipelines to ship releases faster
- Enable automation by executing prompts without any user input locally
- Use API keys, device codes or PKCE to authenticate



brian@example.com

Signed in with AWSIdC | [UserID](#) 

Profile

Profiles are created by your organization admin

KiroProfile-us-east-1 

Estimated Usage

resets on 04/01

Kiro Power

Credits

0.34 used / 10000 covered in plan

0%

 API Keys 

Create and manage API keys for programmatic access to Kiro. Keys are shown only once upon creation.

Enter a name for the new key

Create key

NAME	KEY	CREATED 	
GitHub Actions	ksk_buSBmZml...	Mar 26, 2026	

 Help & Documentation 

Power, flexibility, and security

Familiar environments

Kiro imports VS Code settings in a streamlined AI-ready environment and is also available in the terminal for macOS, Linux and Windows.

Cutting-edge models

Kiro gives you access to frontier and open weight AI models. Pick the right one for the job or let Kiro decide for you by selecting Auto

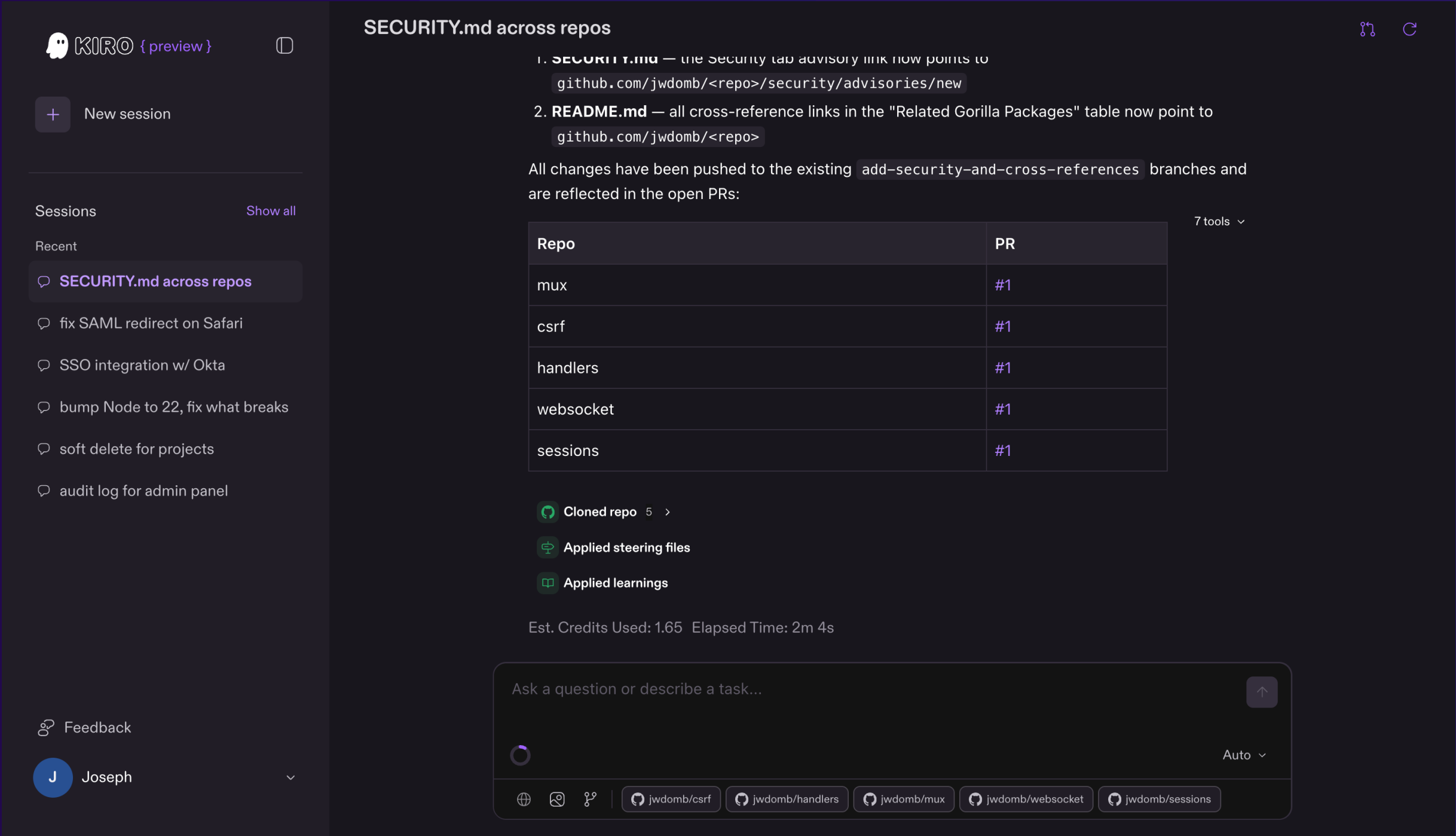
Enterprise-grade security

Kiro is designed, built, and operated by following the same security, governance, and encryption standards as the AWS cloud infrastructure.

Kiro is available in your browser

Build, delegate, and steer from your browser

- **Browser-native surface:** Agentic development experience at app.kiro.dev, no local environment required
- **Same Kiro, new surface:** Steering files, conventions, and credit balance carried over from Kiro IDE and CLI
- **Start with a conversation:** Chat about your codebase, explore approaches, iterate on code together
- **Assign a goal, review the result:** Describe what you need, step away, return to a pull request



Assign a goal, review the result

Autonomous execution with developer oversight

- **Autonomous mode:** Kiro owns the outcome: asks clarifying questions, builds a plan, writes code, delegates to sub-agents, verifies each step, and opens a PR
- **A pull request, not a status update:** Developers review the result, not the process. Nothing merges without explicit approval
- **Context-aware:** Kiro uses brings context from your repositories, web search, and learnings from previous code reviews to every response

Add webhook retry mechanism with exponential backoff

with known deferrals for SSRF, async delivery, 4xx discrimination, etc.).

Review written to: .agents/tasks/task-webhook-retry-mechanism/2026-05-19-155222-review.md

The implementation is complete and all tests pass. Let me verify the state and push.

Clean state on the correct branch. Let me push and create the PR.

Done! I've implemented the webhook retry mechanism and created [Pull Request #3](#).

What was built

Component	Description
WebhookDeliveryService	Core retry engine with exponential backoff (base 1000ms, max 30s) + jitter, configurable max retries per endpoint (default: 5)
DeadLetterService	Circular buffer (1000 entries) for permanently failed deliveries
POST /webhooks/deliver	Accepts { url, payload, maxRetries? }, delivers with retry
GET /webhooks/dead-letter	Returns all dead-letter entries
GET /healthz	Health check endpoint

Verification

- TypeScript strict mode compiles cleanly
- All 23 tests pass (unit + integration)
- Covers retry logic, backoff calculation, dead-letter eviction, input validation, and timeout handling

Est. Credits Used: 10.52 Elapsed Time: 28m 23s

Ask a question or describe a task...

Auto

Autonomous

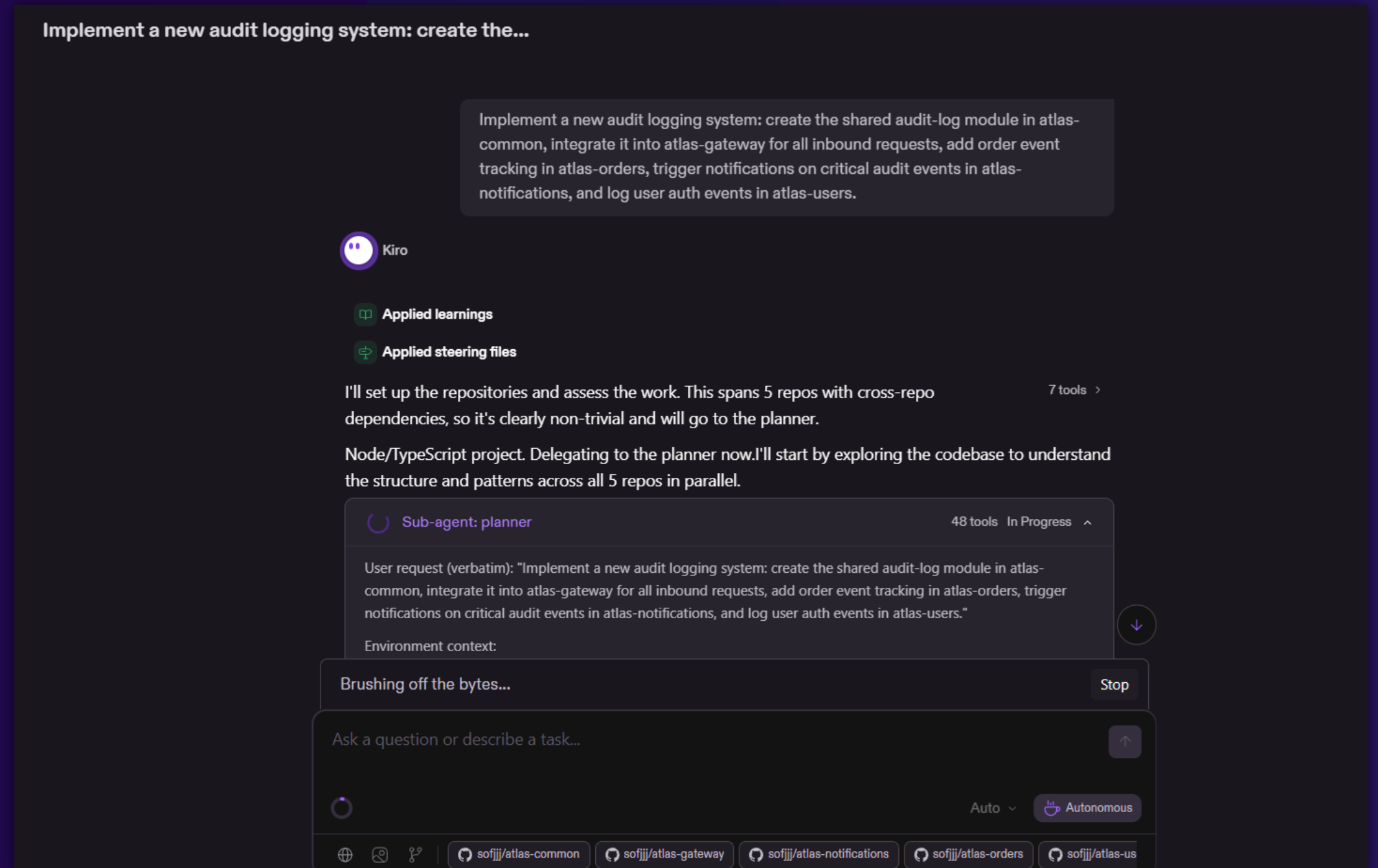
Select repo

sofijj/atlas-gateway

One conversation, multiple repos, coordinated PRs

Changes across repos

- **Select repos at session start:** Choose the repositories involved in the change. Kiro clones each into an isolated sandbox
- **One session, coordinated changes:** Kiro coordinates edits and pull requests across all selected repos in a single conversation
- **Shared libraries → services → clients:** Handles related changes across the stack in one pass



From issue to pull request, sharper over time

GitHub-native workflow

- **Assign from GitHub:** Use a *kiro* label or mention `/kiro` in a comment to assign work directly to the agent
- **Branch, commit, PR:** Kiro creates feature branches, commits on behalf of the developer, and opens PRs
- **Address feedback in place:** `/kiro all` addresses all reviewer comments; `/kiro fix` handles a single thread
- **Steering files:** `.kiro/steering/` loads coding conventions, architecture patterns, and preferences at session start. The same steering carries over across IDE, CLI, and Web
- **Review feedback carries forward:** Feedback from PR comments can shape later work across your repositories

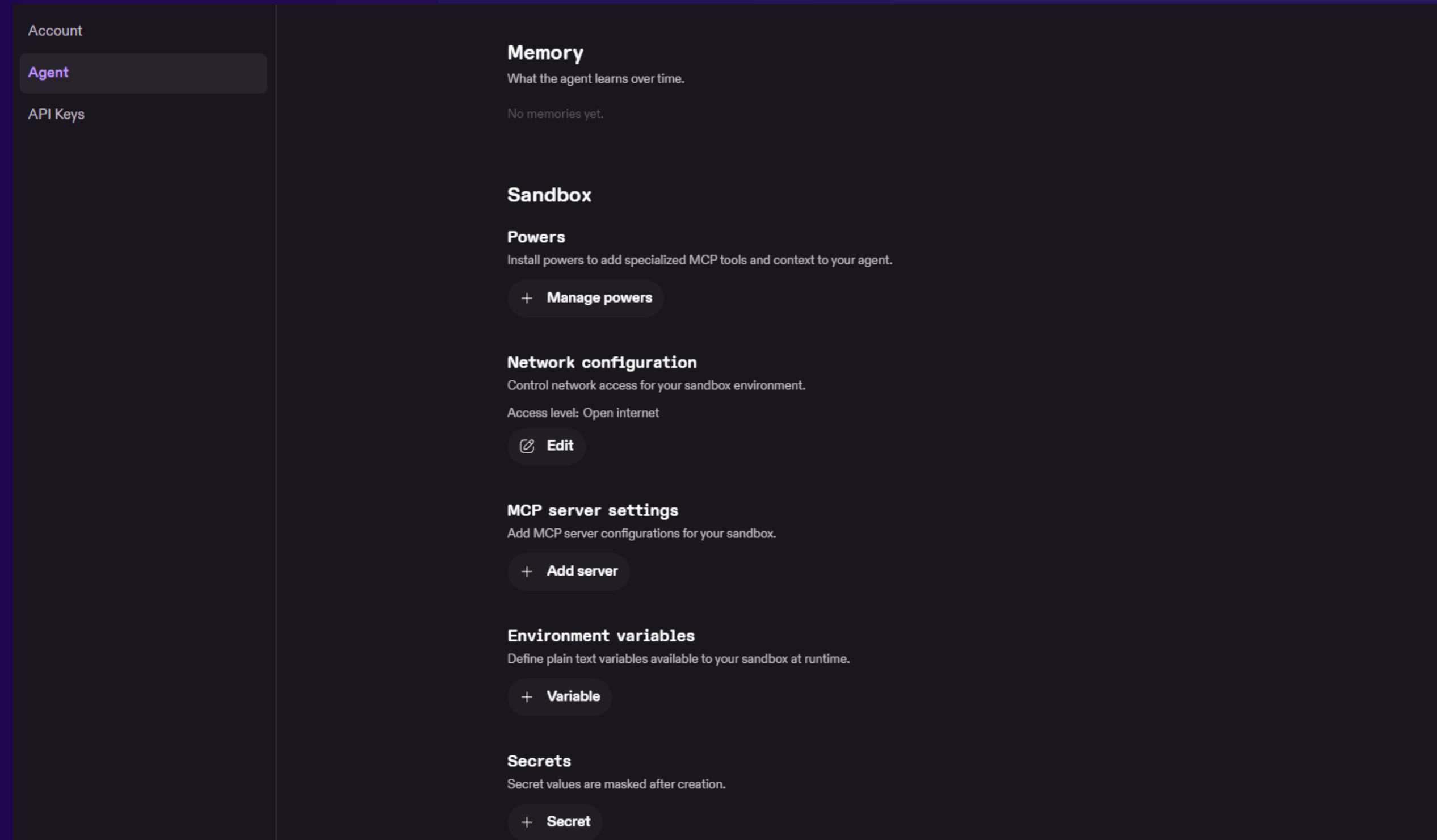
The image displays three overlapping screenshots illustrating the KIRO workflow:

- Top-left screenshot:** A GitHub issue titled "Add webhook retry mechanism with exponential backoff #3". It shows a commit history with a `kiro-agent` commit and a comment from `sofjji` asking to cap the max backoff at 60 seconds.
- Bottom-left screenshot:** A GitHub issue titled "Add webhook retry mechanism with exponential backoff #2". It shows a comment from `sofjji` asking to increase the max backoff cap from 30 seconds to 60 seconds, and a subsequent commit from `kiro-agent` implementing the fix.
- Right screenshot:** A GitHub pull request titled "Add webhook retry mechanism with exponential backoff #3". It shows the pull request details, including the summary, changes, testing results (23 tests passing), and design decisions.

Isolated sandbox with access controls

Cloud sandbox

- **Sandbox per task:** Every session runs in its own isolated sandbox. Kiro clones repos and configures the environment from detected project settings
- **Settings control what the sandbox can reach:** network access, MCP tool configurations, powers, secrets, and base image settings



Kiro now available as a native iOS app

Available in Gated Preview

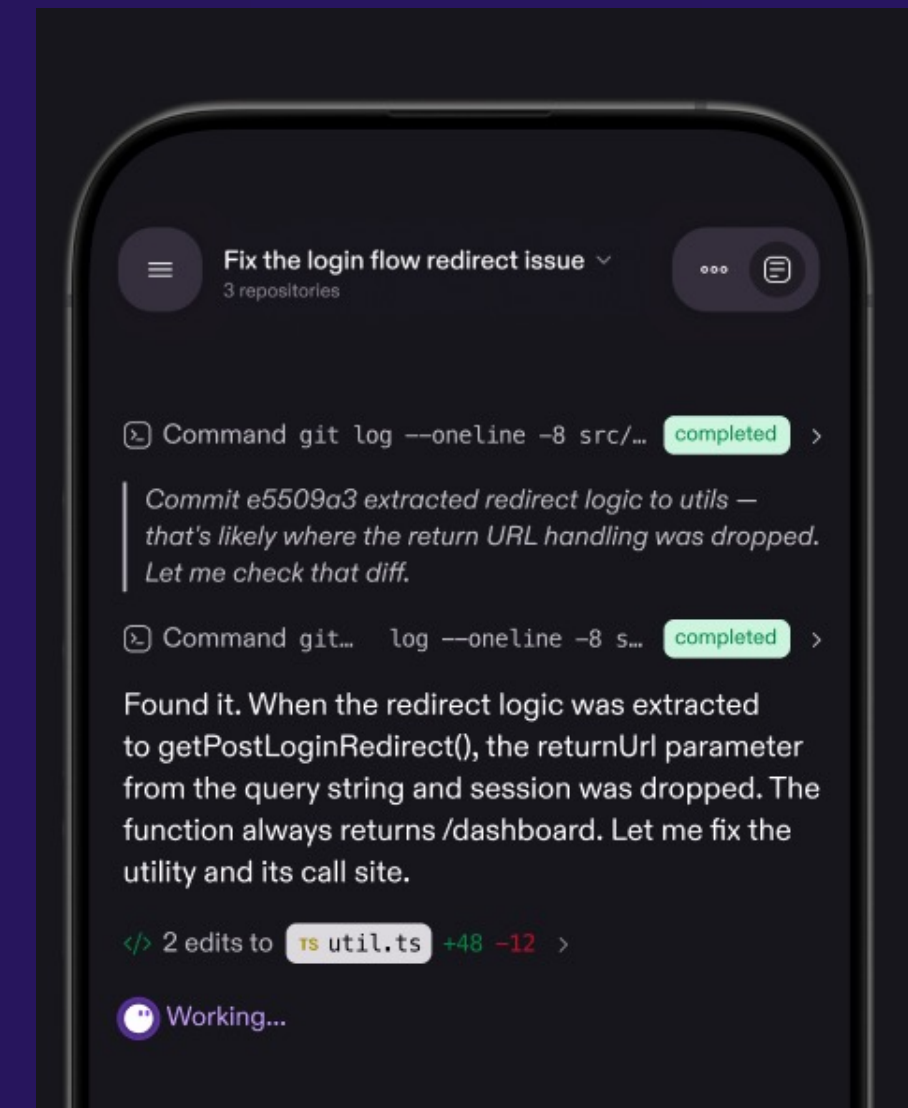
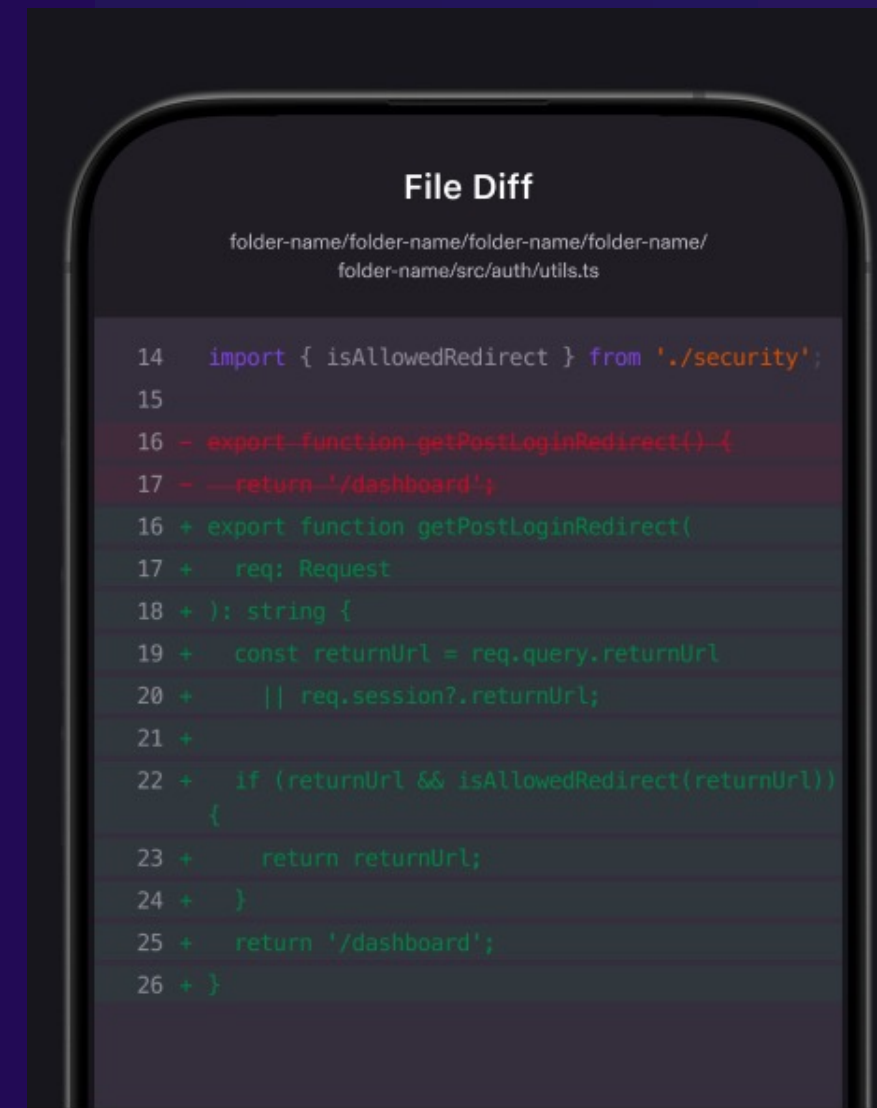
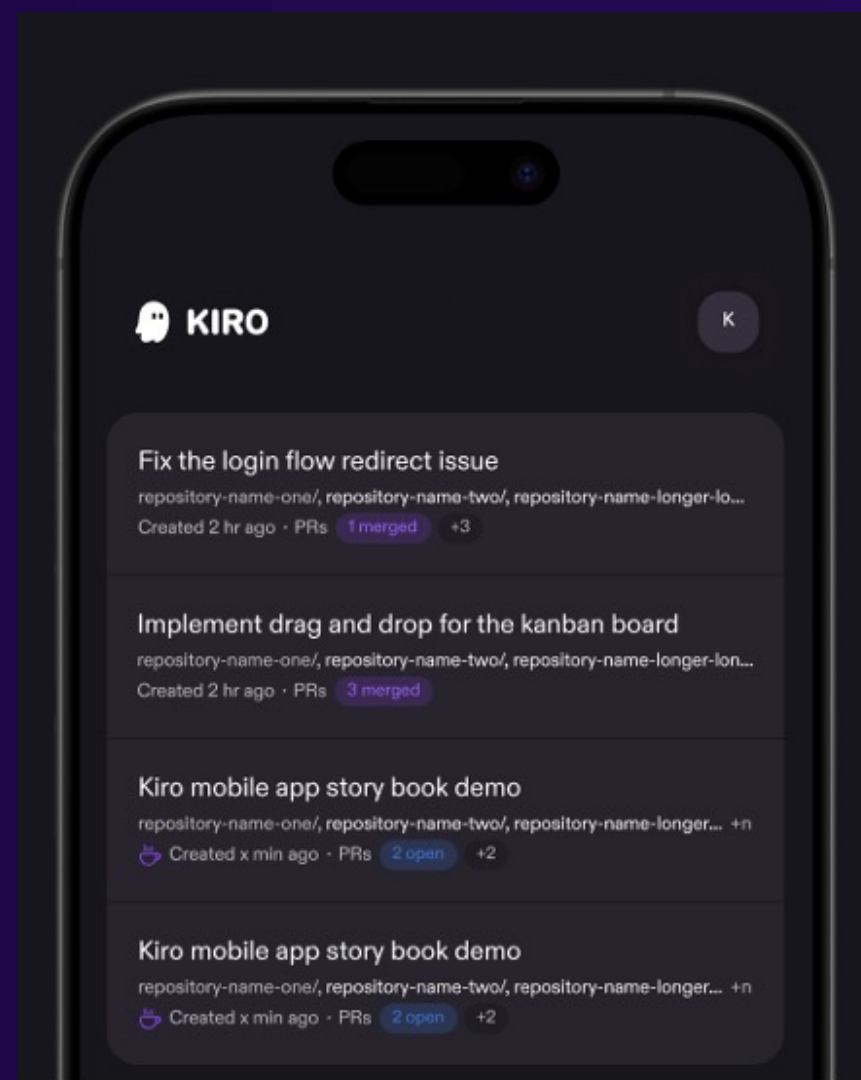
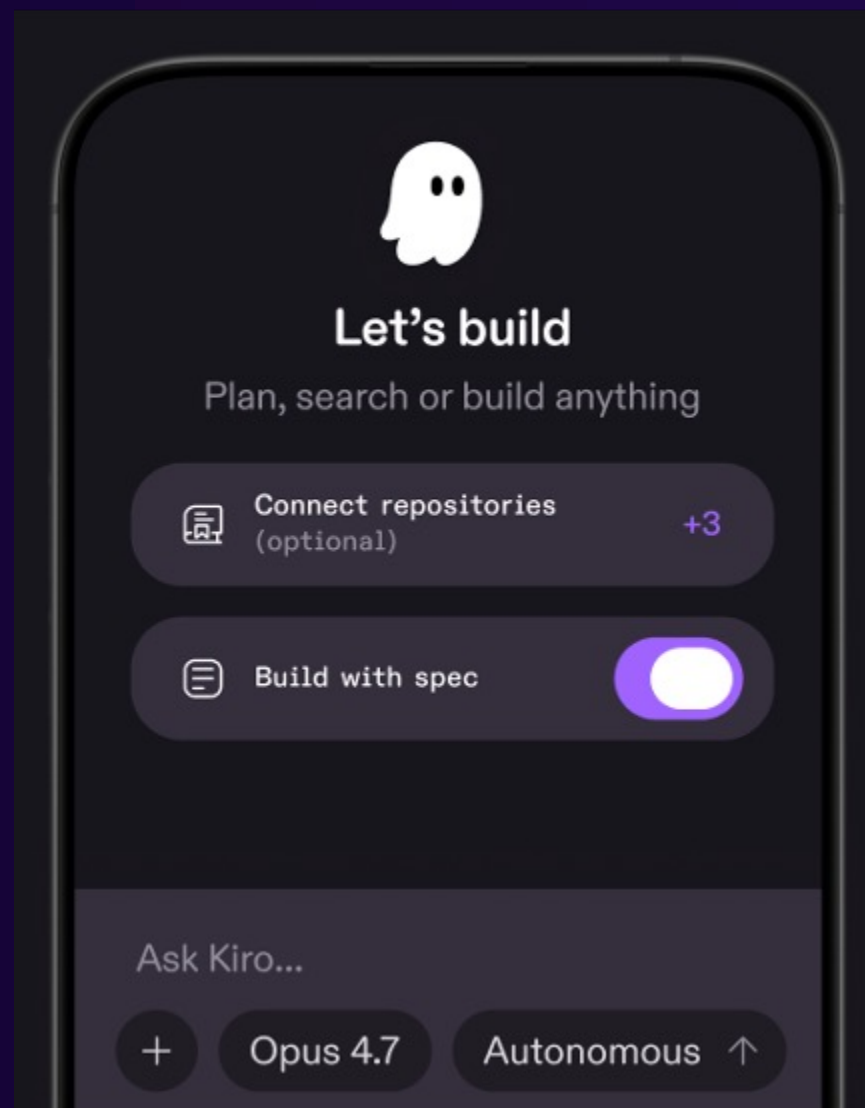
Your Kiro coding sessions, native on iOS.

Run cloud sessions that never stop

Start work, check back later

Diffs built for mobile

Same agent, same context





Introducing Kiro powers

Extend Kiro agent capabilities through sharable best practices





Gracias

Conectemos

Información de contacto

Email: hdomingy@amazon.com

Linkedin: [in/hdominguezy](https://www.linkedin.com/in/hdominguezy)

Información de contacto

Email: andiaa@amazon.com

Linkedin: [in/arepageek](https://www.linkedin.com/in/arepageek)